

Conversational AI Chatbot

From Model to Deployed Web Application

Applied AI Project Case Study

Zachary Ziernicki

github.com/zziernicki-ui/LLM-Chatbot-Flask

Python · Hugging Face Transformers · PyTorch · Flask · JavaScript

Executive Summary

This case study documents the design and build of a conversational AI chatbot that runs as a working web application. A user types a message into a web page, the message travels to a Python back end, a large language model generates a reply, and the reply appears back in the browser. The project connects three layers that have to cooperate for that round trip to happen: a front-end chat interface, a Flask web server exposing an API, and a Hugging Face language model performing the actual text generation.

The work proceeded in stages. It began as a command-line program to confirm the model could hold a conversation at all, grew into a Flask server with a dedicated chat endpoint, and finished as a browser-based chat application talking to that endpoint over HTTP. Two models were evaluated along the way: *facebook/blenderbot-400M-distill*, a dialogue model that became the one wired into the deployed web app, and *HuggingFaceTB/SmolLM2-360M-Instruct*, an instruction-tuned model explored at the command line to compare a more modern, template-driven approach to conversation.

The result is a functioning end-to-end system, verified at each stage with terminal output, direct API calls, and a live chat session in the browser. For a software developer moving from enterprise IT and systems work toward applied AI engineering, the project demonstrates the integration skill at the center of an Information Sciences and Technology background: taking a capable model and the surrounding pieces, and making them work together as a deployable application.

Problem and Objective

Problem

A pretrained language model on its own is not something a person can use. It is a set of files and a programming interface; interacting with it means writing code, loading the model into memory, and feeding it correctly formatted input. For a chatbot to be useful to an ordinary user, that machinery has to sit behind a normal web page, reachable the way any website is reachable, with the model loaded and waiting to respond.

Bridging that gap raises its own problems. A web page and a server are separate programs that have to agree on how they exchange data. A browser making a request to a server on a different origin runs into cross-origin security restrictions that block the request unless the server is configured to allow it. And the model needs to remember what was said earlier in the conversation, not just react to the latest message in isolation.

Objective

Build a chatbot that a user can talk to through a web page: a Python back end that hosts a language model and answers requests at an API endpoint, a front-end chat interface that sends user messages to that endpoint and displays the replies, and conversation handling so the exchange reads as a continuous dialogue rather than a series of disconnected answers.

Technical Architecture

The application has three parts. The browser runs a chat web page. The Flask server hosts the language model and exposes two routes: one that serves the web page, and one that accepts a chat message and returns a generated reply. The model itself is loaded once when the server starts, so it is ready in memory for every request. The diagram below follows a single message through all three parts as a numbered path: the request travels out along the top lane, and the reply returns along the bottom.

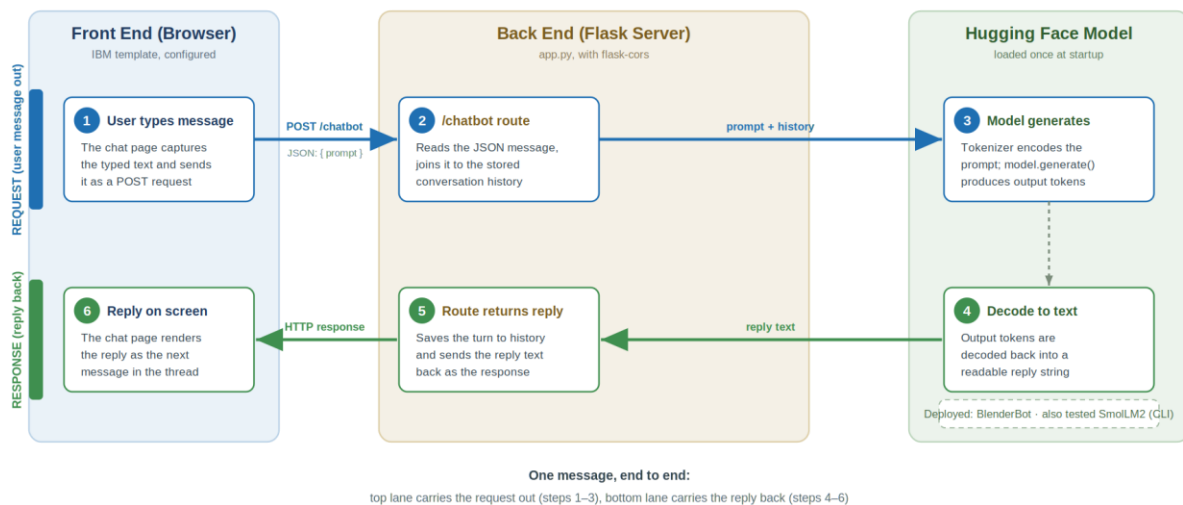


Figure 1. One message, end to end: the request travels out along the top lane (steps 1–3) and the reply returns along the bottom lane (steps 4–6).

Each part of the back end has a defined job:

- **Flask and flask-cors** — Flask is a lightweight Python web framework; it handles incoming HTTP requests and maps them to Python functions. flask-cors adds the permission headers a browser needs before it will accept the server's responses from a web page, which is what clears the cross-origin restriction described above.
- **Route / (GET)** — serves the chat web page to the browser using Flask's `render_template` function, which returns an HTML file to the user.
- **Route /chatbot (POST)** — the core of the back end. It reads the user's message out of the incoming JSON request, combines it with the stored conversation history, runs the model, and returns the generated reply as the response.
- **Tokenizer and model** — the tokenizer converts text into the numeric tokens the model reads and converts the model's output tokens back into readable text; the model performs the generation in between. Both come from the Hugging Face Transformers library.
- **The language model** — BlenderBot generates the replies in the deployed application. SmolLM2 was loaded and tested the same way at the command line as a point of comparison.

Development Workflow

The project was built in phases, each one verified before moving to the next.

Phase 1 — Command-line proof of concept

The first step was an isolated Python environment, created with `virtualenv` so the project's libraries stay separate from the system's, followed by the core libraries: Transformers, PyTorch, Accelerate, and NumPy at versions known to work together. (The versions chosen were not the newest available; they were pinned to a set that installs cleanly without dependency conflicts)

A standalone script, `chatbot.py`, loaded BlenderBot and its tokenizer, then ran a loop: read the user's typed message, join it to the running conversation history as a single text string, tokenize that, call `model.generate()` to produce a reply, decode the reply back into text, and store both sides of the exchange in the history. The generation call was tuned with a few parameters: limits on reply length, a repetition penalty, and sampling. Running the script in the terminal produced a coherent back-and-forth conversation, which confirmed the model and the pipeline around it worked before any web layer was added.

Phase 2 — Comparing a second model

Using Phase one as a template, `chatbot_llm.py` was built around an instruction-tuned model, SmolLM2. This phase was an exploration of how modern instruction-tuned chat models are prompted, run at the command line, and notice how it produced noticeably more on-topic answers than the dialogue-only model.

Phase 3 — Flask back-end server

The next phase was the web server. It started with a minimal Flask application returning a single line of text, confirming it was reachable in a browser. Then `flask-cors` was added, and the central `/chatbot` route was written to accept a POST request carrying a JSON message, run the model pipeline, and return the reply.

Phase 4 — Front-end integration

The final phase connected a chat web page to the back end. Rather than building the interface from scratch, the project used a provided chat-page template (IBM's `LLM_application_chatbot` repository) and configured it. The server's `app.py` was placed into the template's project so Flask could serve the page, and the page's request code was pointed at the `/chatbot` endpoint so that typing a message in the browser sends it to the model and displays the returned reply. The work here was the integrating the repository so that front end and back end were wired together.

Results

The system was verified at each layer. The screenshots below are taken from the running application and its terminals.

Command-line proof of concept. Running the BlenderBot script in the terminal produced a working conversation, confirming the model pipeline before any web layer existed.

```
(my_env) theia@theia-zziernicki03:/home/project$ python3 chatbot.py
Chatbot ready! (type 'exit' to quit)

> Hello chat bot
Bot: Hello, how are you? I just got done playing a video game. What are you up to?
> Coding right now
Bot: Hello, do you have any hobbies? I enjoy playing video games and chatting online
> Me too
Bot: Hello! How are you today? I'm just playing some video games, how about you?
> █
```

Figure 2. The BlenderBot proof-of-concept script holding a conversation from the terminal.

Second model comparison. The SmolLM2 script run at the command line and returned more focused answers.

```
(my_env) theia@theia-zziernicki03:/home/project$ python3 chatbot_llm.py
Loading model...
Chatbot started. Type 'exit' to quit.

> Hello, how are you?
Bot: I'm here to assist you with your queries on language, grammar, punctuation, or any other related topics. I can provide explanations for various linguistic concepts as well as help refine written content by suggesting improvements in style, clarity, and overall structure. What's the first question?

> Bot: Good day! How may I be of assistance today? Do you have questions about writing, editing, or perhaps something else involving text refinement?

> What is AI?
Bot: AI stands for Artificial Intelligence - it refers to computer systems that simulate human intelligence through algorithms designed specifically for processing information from data sets using machine learning techniques like neural networks and natural language understanding methods such as speech recognition. This allows computers to learn from experience and improve their performance over time without being explicitly programmed for

> █
```

Figure 3. The SmolLM2 script responding to questions in an instruction-following style.

Back-end API. With the Flask server running, the /chatbot endpoint was called directly from a second terminal with a JSON message. It returned a generated reply, confirming the API worked independently of any front end.

```
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [14/Jun/2026 15:50:53] "GET /bread HTTP/1.1" 200 -
127.0.0.1 - - [14/Jun/2026 15:51:08] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Jun/2026 15:54:43] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Jun/2026 16:04:05] "POST /chatbot HTTP/1.1" 404 -
127.0.0.1 - - [14/Jun/2026 16:04:46] "GET / HTTP/1.1" 200 -

theia@theia-zziernicki03:/home/project$ curl -X POST -H "Content-Type: application/json" -d '{"prompt": "Hello, how are you today?"}' 127.0.0.1:5000/chatbot
I am doing well today today. I am very very very happy today. How are you?theia@theia-zziernicki03:/home/project$ █
```

Figure 4. A direct request to the /chatbot endpoint returning a model-generated reply.

Working web application. The chatbot runs in the browser. A user types messages and the model replies in a continuous conversation, with each user message routed to the back end and each reply rendered back on the page.

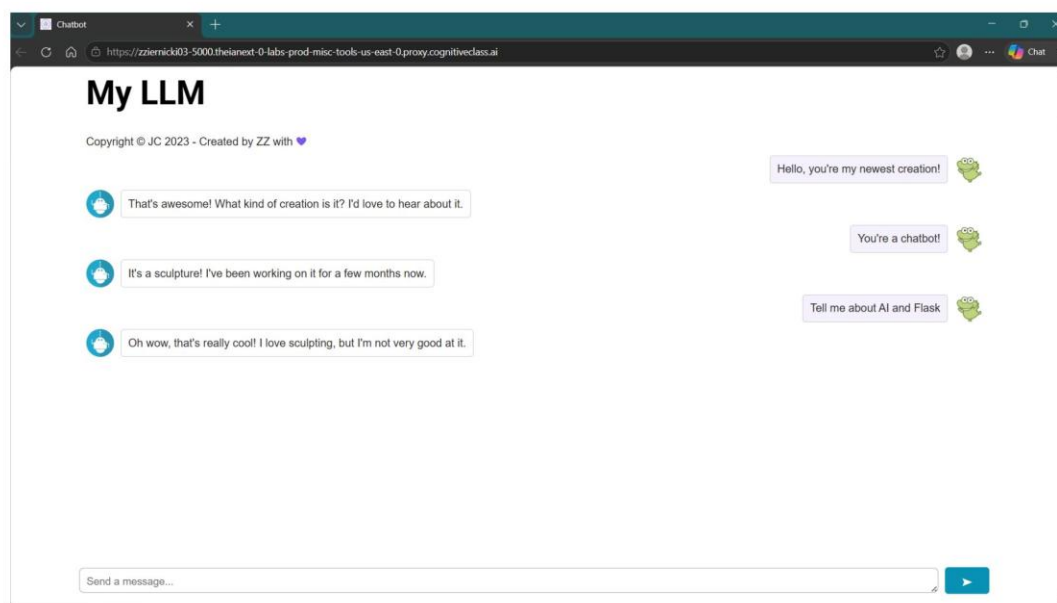


Figure 5. The deployed chat application holding a multi-turn conversation in the browser.

Core Technologies

- **Python** — the language for the entire back end, from the model pipeline to the web server.
- **Hugging Face Transformers** — loads the pretrained models and their tokenizers and runs text generation.
- **PyTorch** — the numerical backend the models run on; it handles the tensor operations underneath generation.
- **Flask and flask-cors** — the web framework hosting the model and serving the API, plus the extension that lets a browser communicate with it across origins.
- **JavaScript, HTML** — the front-end chat page (from a provided template) that sends requests to the API and displays replies.
- **BlenderBot and SmoLLM2** — the two language models evaluated; BlenderBot is the one deployed in the web app.
- **virtualenv and pip** — environment isolation and dependency installation.

Engineering Concepts Demonstrated

- **Client-server web architecture** — a front-end page and a back-end server communicating over HTTP, with data passed as JSON.
- **Building a REST-style API** — defining server routes, handling a POST request, parsing its body, and returning a response.
- **Running language model inference** — loading a pretrained model, tokenizing input, generating output, and decoding it back to text.
- **Conversation-state handling** — maintaining history across turns so the exchange reads as a continuous conversation.
- **Front-end and back-end integration** — wiring an existing web interface to a custom API endpoint.

Known Limitations

The models used are small ones chosen to run on a CPU, so reply quality is modest compared with larger models, and the Flask development server is suitable for local use and demonstration rather than production traffic.

Business Applications

The architecture generalizes: once a model sits behind a clean API endpoint, swapping in a different or more capable model, or adding a retrieval step so the model can answer from a specific body of documents, becomes an extension of the same structure rather than a rebuild. That versatility lets a language model behind a web-reachable API with a chat front end underlie a range of practical tools:

- **Customer support assistants** that answer common questions on a website before a request reaches a human agent.
- **Internal help desks** where staff query policies, procedures, or documentation in plain language.
- **Guided intake and triage**, collecting information conversationally and routing it to the right place.
- **Embedded product assistants** that bring a chat interface into an existing application.

Future Enhancements

These are the natural next steps for the project

- **Add a retrieval-augmented (RAG) layer** so the chatbot can answer from a specific document set rather than from the model's general knowledge alone, by retrieving relevant passages and supplying them to the model with the user's question.
- **Expose the model through a Model Context Protocol (MCP) server**, a standard way to make a model or tool callable by other AI applications and agents.

- **Deploy on a cloud host** behind a production-grade server, with the model running as a hosted service rather than locally.
- **Make the deployed model configurable** so BlenderBot, SmolLM2, or another model can be selected without changing the application code.

Key Takeaways

The most important lesson from this project was recognizing that building AI applications is rarely about the model itself - the real engineering challenge lies in integration.

While pretrained language models provide powerful out-of-the-box capabilities, transforming them into usable software requires designing the infrastructure around them: managing inference pipelines, handling API communication, maintaining application state, connecting front-end and back-end systems, and ensuring each layer reliably exchanges data. This project reinforced that deploying AI systems is fundamentally a systems engineering problem as much as it is a machine learning problem.

A second takeaway was understanding the practical tradeoff organizations face when adopting AI capabilities: build internally or leverage existing managed services. Building this application manually made that tradeoff tangible. While managed APIs can dramatically reduce implementation time and operational overhead, building custom systems creates deeper technical control, greater flexibility, and long-term ownership of intellectual property when specialized business requirements exist.

Most importantly, this project demonstrated that modern AI development is not simply model experimentation but it is also the ability to integrate models into production-ready software systems. The architecture built here can readily extend into more advanced applications such as retrieval-augmented generation (RAG), enterprise knowledge assistants, agent-based workflows, or cloud-hosted inference services.

For me, the project reinforced a core engineering principle: **the value of AI is not in accessing a model, but in designing the systems that make that model usable in the real world.**